Decision Trees

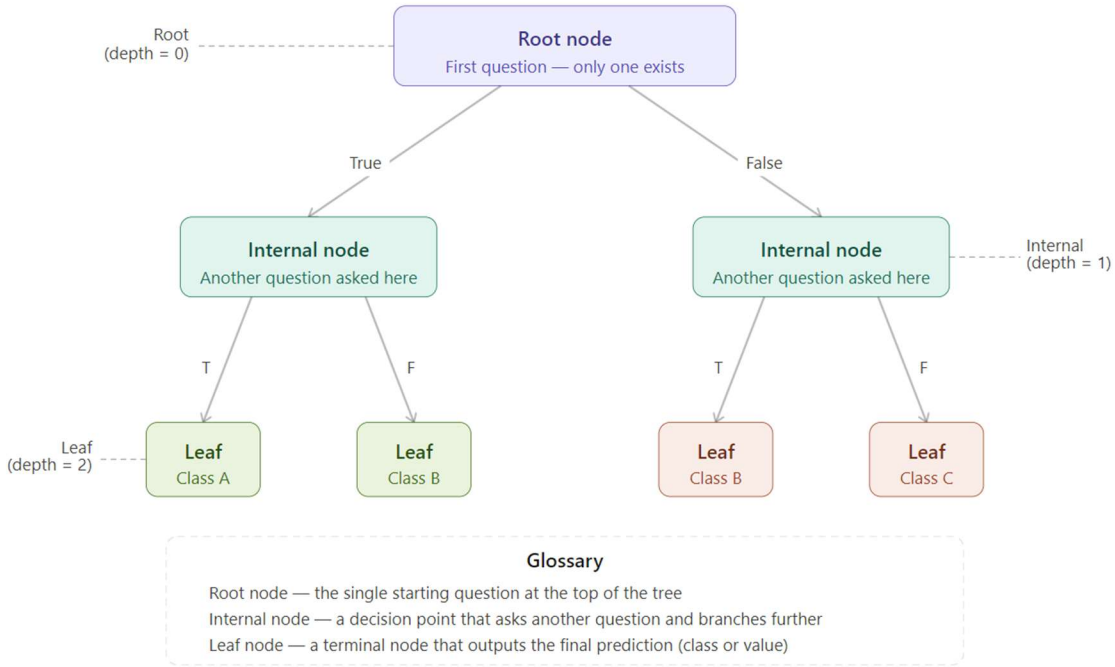
1. What is a Decision Tree

A decision tree is a supervised machine learning model that makes predictions by asking a series of questions about the input data. Each question splits the data into smaller groups, and the process continues until a final answer is reached.

Think of it like a flowchart. You start at the top, answer yes or no to a question, move to the next question, and eventually land on a final result.

2. Anatomy of a Decision Tree

Every decision tree is made up of three types of nodes. Understanding each one is essential before learning how the tree learns or makes decisions.



2.1 Root Node

- **What it is:** The single starting point at the very top of the tree.
- **Depth:** 0 (it sits at the highest level).
- **What it does:** Asks the first question and splits all the data based on the answer.
- **How many:** There is always exactly one root node per tree.

2.2 Internal Node

- **What it is:** Any node that sits between the root and the leaves.
- **Depth:** 1 or deeper (depending on how many levels the tree has).
- **What it does:** Asks another question and further splits the data coming into it.
- **How many:** A tree can have multiple internal nodes, one for each branching decision.

2.3 Leaf Node

- **What it is:** The final node at the bottom of a branch, also called a terminal node.
- **What it does:** Outputs the final prediction, either a class label (classification) or a numeric value (regression).
- **No further branching:** A leaf never asks another question.

Key Point: Every data point that enters the tree travels from the root, through internal nodes, and always ends at exactly one leaf.

3. Impurity: Gini and Entropy

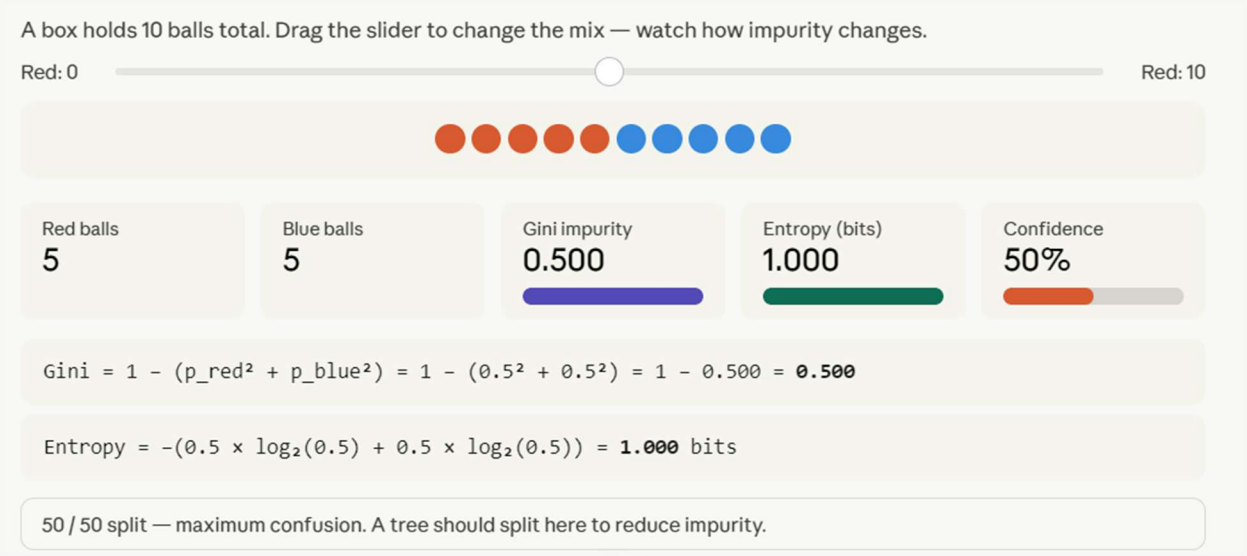
Before a tree can decide where to split, it needs a way to measure how mixed or confused a group of data is. This is called impurity.

A node is pure if it contains only one class. A node is impure if it contains a mix of classes. The goal of a decision tree is to create splits that reduce impurity as much as possible.

3.1 Gini Impurity

Definition: A measure of how often a randomly chosen item from a node would be incorrectly classified.

- **Range:** 0 to 0.5 for binary classification (0 to 1 in the general form).
- **Gini = 0:** Perfectly pure node. All items belong to one class.
- **Gini = 0.5:** Maximum impurity. Items are split 50/50 between two classes.



Try this in the app for better understanding.

Formula: Gini = 1 minus the sum of the squared proportions of each class.

Example with 5 red and 5 blue balls (10 total):

p_red = 0.5, p_blue = 0.5

Gini = 1 - (0.5^2 + 0.5^2) = 1 - 0.500 = 0.500

This is maximum confusion. The tree would strongly want to split here.

3.2 Entropy

Definition: A measure from information theory that quantifies the unpredictability or disorder in a group.

- **Range:** 0 to 1 bit for binary classification.
- **Entropy = 0:** Perfectly pure. No uncertainty at all.
- **Entropy = 1:** Maximum uncertainty. Both classes are equally likely.

Formula: Entropy = negative sum of each proportion multiplied by log base 2 of that proportion.

Example with 5 red and 5 blue balls:

Entropy = -(0.5 x log2(0.5) + 0.5 x log2(0.5)) = 1.000 bit

Gini vs Entropy: Both measure impurity and usually give very similar results. Gini is slightly faster to compute. Entropy can sometimes be more sensitive to class imbalance. In practice, the choice between them rarely changes the final tree significantly.

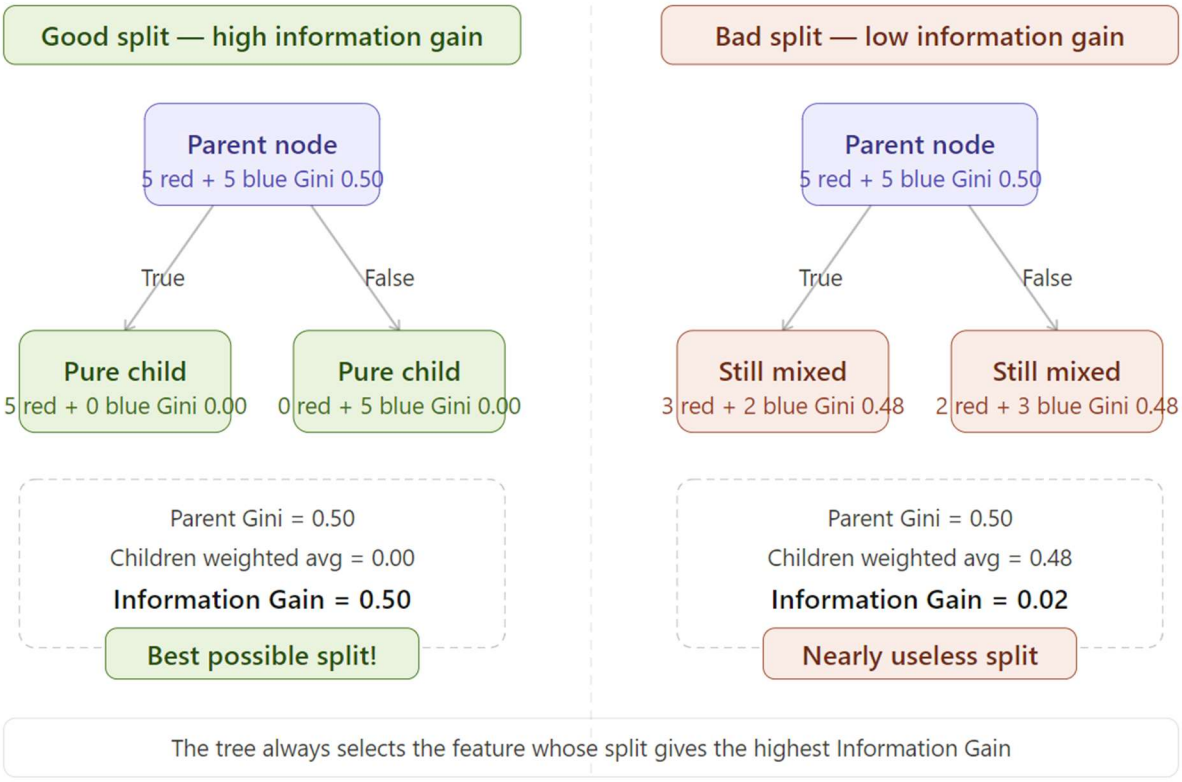
4. Information Gain

Information Gain is the key metric a decision tree uses to choose which feature to split on at each node.

Definition: The reduction in impurity achieved by splitting on a particular feature.

Formula: Information Gain = Parent Impurity minus the weighted average impurity of the child nodes.

The tree evaluates every available feature and every possible split point. It then selects the split that gives the highest Information Gain.



4.1 Good Split: High Information Gain

Starting condition: Parent node has 5 red and 5 blue balls. Gini = 0.50.

After the split:

- True child gets 5 red and 0 blue. Gini = 0.00 (pure).
- False child gets 0 red and 5 blue. Gini = 0.00 (pure).

Weighted average of children = 0.00

Information Gain = 0.50 - 0.00 = 0.50 (the maximum possible)

This is the best possible split. Both children are completely pure after the split.

4.2 Bad Split: Low Information Gain

Starting condition: Same parent with 5 red and 5 blue balls. Gini = 0.50.

After the split:

- True child gets 3 red and 2 blue. Gini = 0.48 (still mostly mixed).
- False child gets 2 red and 3 blue. Gini = 0.48 (still mostly mixed).

Weighted average of children = 0.48

Information Gain = 0.50 - 0.48 = 0.02 (nearly useless)

This split barely changes anything. The tree would reject this feature in favour of one that produces a higher gain.

Key Rule: The tree always picks the feature and threshold that maximises Information Gain at each step.

5. How the Tree Grows

At each node, the algorithm does the following:

- Looks at every feature in the dataset.
- Tries every possible threshold or split point for each feature.
- Calculates the Information Gain for each candidate split.
- Selects the split with the highest Information Gain.
- Creates two child nodes and repeats the process on each subset.

The tree stops growing when one of these conditions is met:

- A node becomes pure (Gini = 0 or Entropy = 0).
- A maximum depth limit is reached.
- The number of samples in a node falls below a minimum threshold.
- The Information Gain from any further split is too small to be useful.

Here is a streamlined, interview-optimized version of the Random Forest notes. The information has been structured to highlight the "what" and the "why," which is exactly how technical interviewers expect candidates to explain these concepts.

Ensemble learning in machine learning combines multiple individual models (called "base models" or "weak learners") into a single, comprehensive predictive model. Instead of relying on one algorithm, this method aggregates their results to reduce errors, prevent overfitting, and drastically improve overall accuracy and robustness

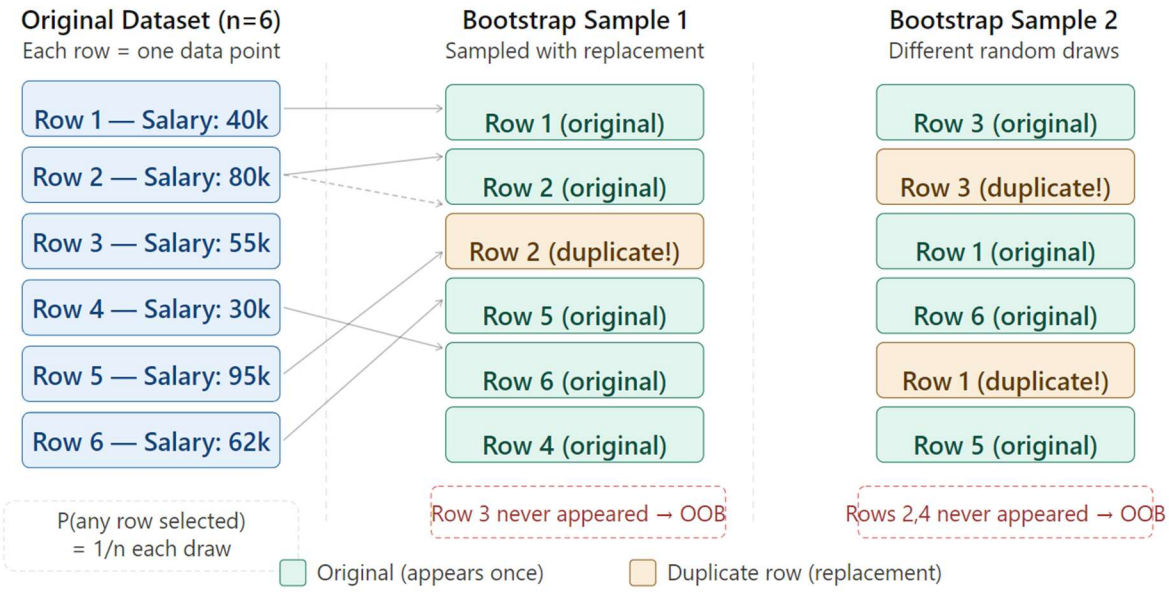
Random Forest

1.Bootstrapping

The Concept: Bootstrapping is a statistical resampling technique where "n" samples are randomly drawn from a dataset of size "n", with replacement.

Interview Talking Points:

- A drawn row is placed back into the pool, meaning some rows appear multiple times while others never do.
- The probability of a specific row not being selected in a single draw is 1 - (1/n).
- Over n draws, as n approaches infinity, the probability of a row never being selected converges to e^-1 ≈ 0.368.
- On average, each bootstrap sample contains ~63.2% of the original rows, leaving ~36.8% out.
- **The "Why":** It introduces controlled randomness, giving each tree a slightly different data distribution to ensure diverse decision-making.



2. Bagging (Bootstrap Aggregating)

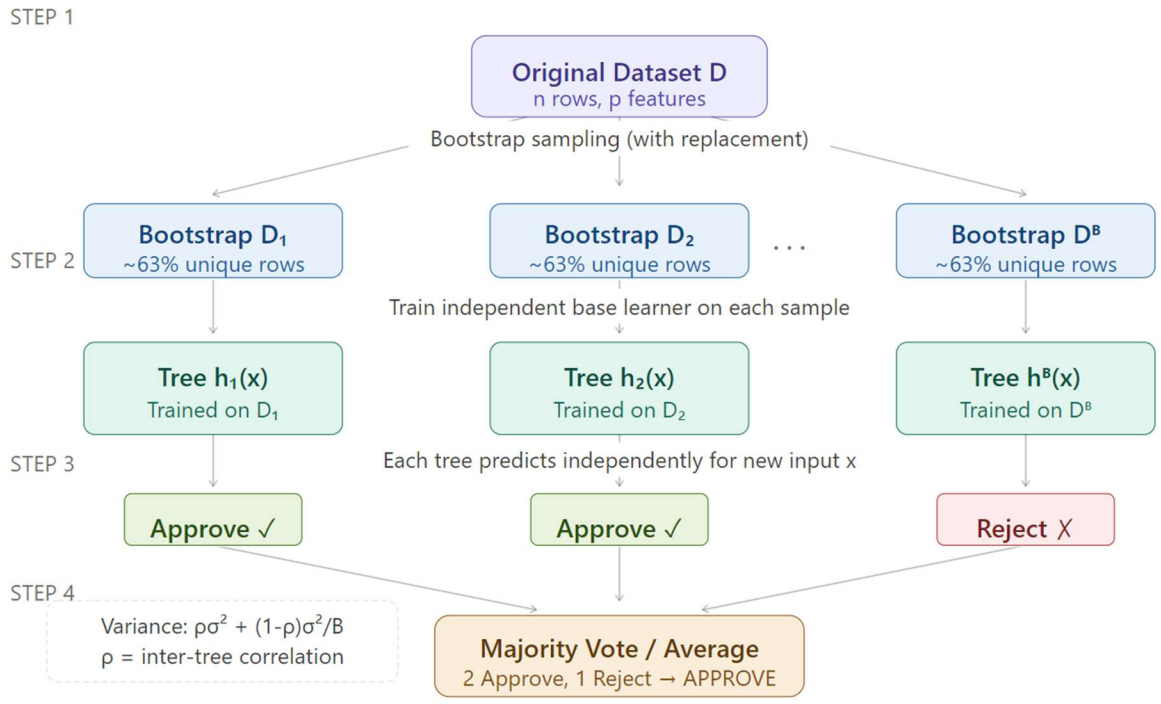
The Concept: Bagging is an ensemble method that trains multiple base learners independently on separate bootstrap samples and combines their predictions.

Interview Talking Points:

- Predictions are combined via majority voting for classification or averaging for regression.
- Bagging purely reduces model variance, not bias.
- It works exceptionally well with high-variance, low-bias models like fully-grown Decision Trees.
- Adding more trees does not cause overfitting, though computational returns diminish after a few hundred trees.
- **The "Why" (The Math):** Bagging mathematically reduces variance based on this formula:

$Var = \rho \cdot \sigma^2 + (1 - \rho) \cdot (\sigma^2 / B)$

- In this equation, ρ (rho) is the inter-tree correlation, σ^2 (sigma squared) is the variance of a single tree, and B is the total number of trees.

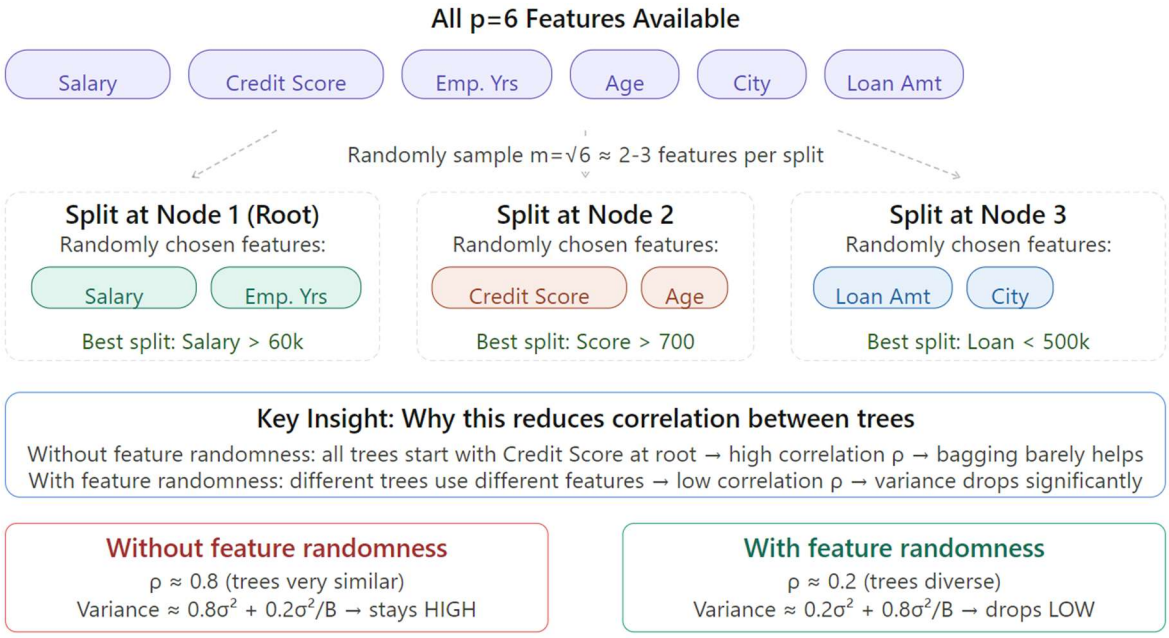


3. Random Subset of Variables (Feature Randomness)

The Concept: Instead of evaluating all "p" features to find the best split, only a random subset of "m" features is considered.

Interview Talking Points:

- Feature randomness operates dynamically at the individual node level, not at the tree level.
- For classification tasks, the standard subset size is $m = \sqrt{p}$.
- For regression tasks, the standard subset size is $m = p / 3$.
- **The "Why":** Without this, the strongest predictive feature would dominate the root nodes of all trees, making them structurally identical.
- Forcing feature randomness directly reduces the inter-tree correlation (ρ), which is required for bagging to effectively shrink overall variance.

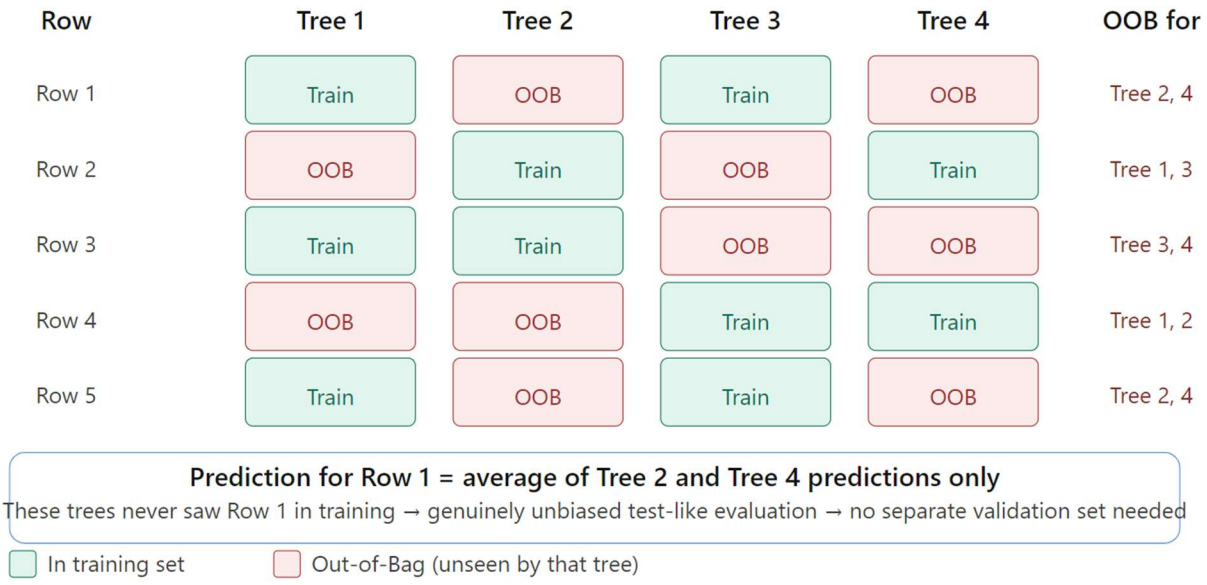


4. Out-of-Bag (OOB) Data & Error Rate

The Concept: The ~36.8% of rows not selected during the bootstrapping process for a specific tree act as its Out-of-Bag (OOB) data.

Interview Talking Points:

- OOB data serves as a free, built-in validation set.
- The OOB prediction for a specific row is made solely by aggregating predictions from the trees that never saw that row during training.
- The OOB Error Rate estimates generalization error without requiring a separate holdout set.
- This evaluation operates with zero additional training cost, giving it an efficiency advantage over K-Fold Cross-Validation.
- The error estimate is considered highly reliable for large forests containing 200 or more trees.

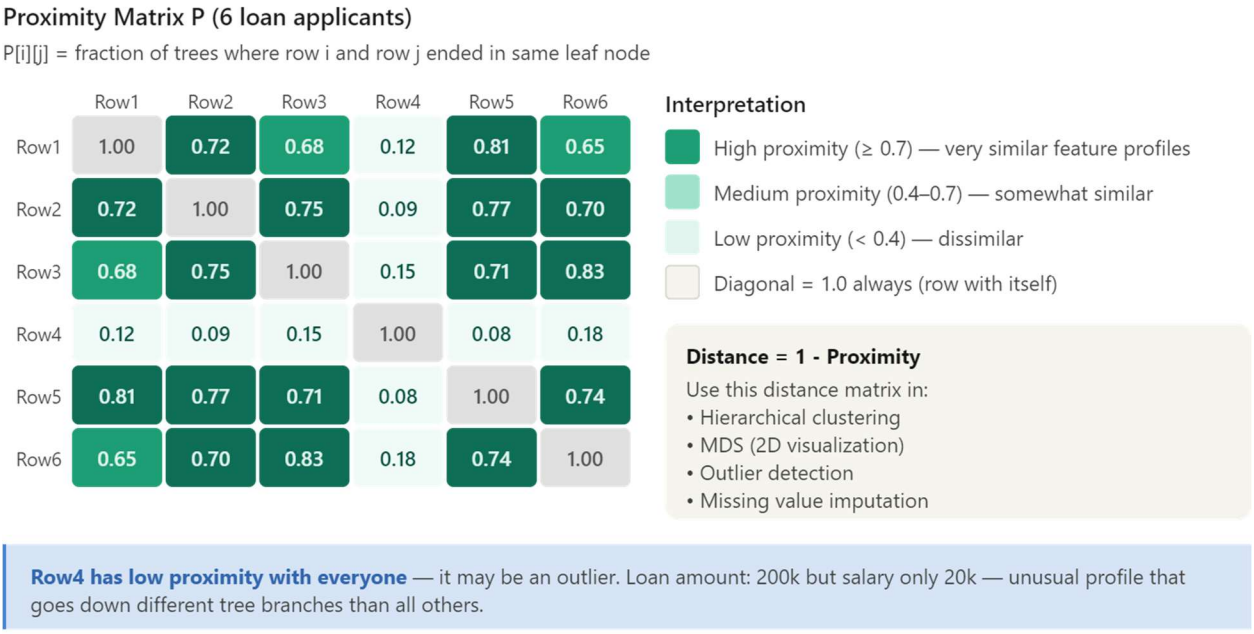


5. Proximity Matrix

The Concept: An n × n matrix tracking the structural similarity of observations based on how frequently they end up in the same terminal leaf node across the forest.

Interview Talking Points:

- A matrix value of 1.0 means two observations share a leaf node in every single tree, while 0.0 means they never do.
- **Use Cases:** It enables powerful unsupervised tasks like outlier detection, data visualization via Multidimensional Scaling, missing value imputation, and prototype selection.
- **Trade-offs:** Computing the full matrix is memory-intensive and computationally expensive, requiring $O(n^2)$ memory and $O(n^2B)$ operations.



Quick Recall Cheat Sheet

Concept	Core Mechanism	Interview "Why it Matters"
Bootstrapping	Samples rows with replacement.	Injects diversity into training distributions.
Bagging	Aggregates independent models.	Shrinks overall model variance.
Feature Randomness	Splits nodes on random feature subsets.	Lowers inter-tree correlation for better bagging.
OOB Data & Error	Evaluates trees on unseen ~36.8% data.	Provides a free, highly accurate validation metric.
Proximity Matrix	Tracks co-leaf frequency across trees.	Unlocks clustering, imputation, and outlier detection.